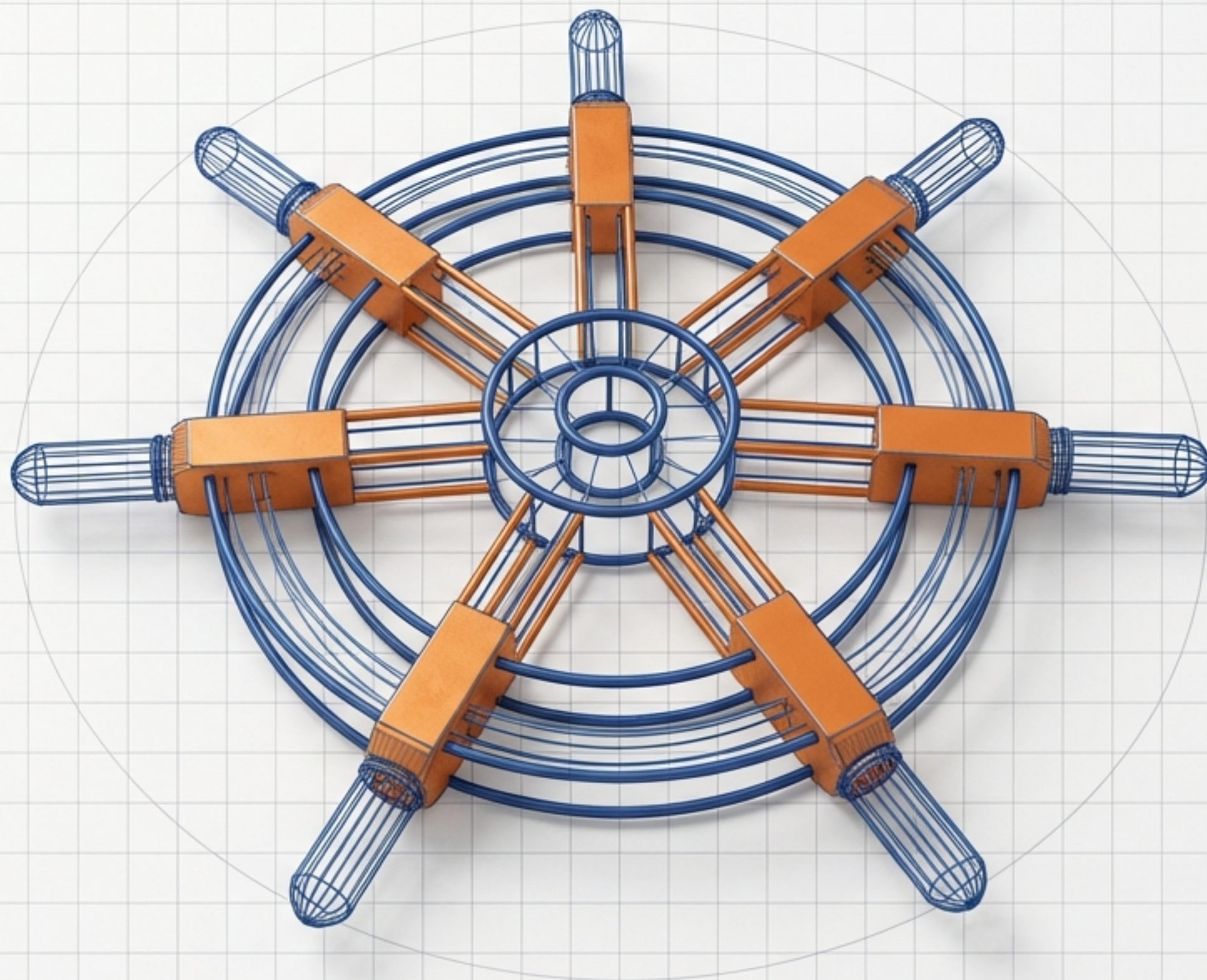


Giải mã Kubernetes

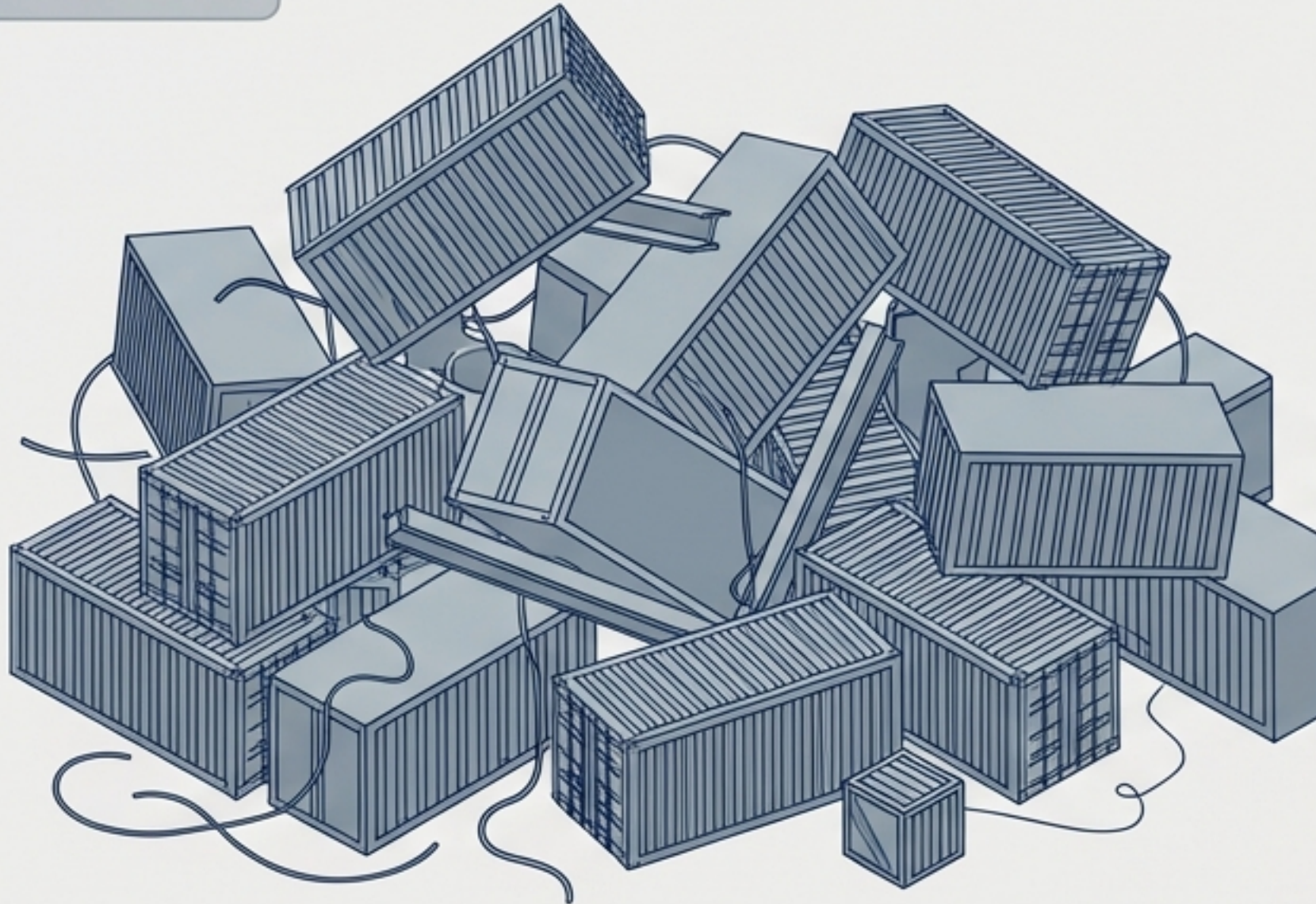
Bản đồ tư duy toàn diện về Hệ sinh thái Điều phối Container. Khám phá kiến trúc, luồng vận hành và các thành phần cốt lõi của nền tảng quản lý hạ tầng đám mây mạnh mẽ nhất thế giới.



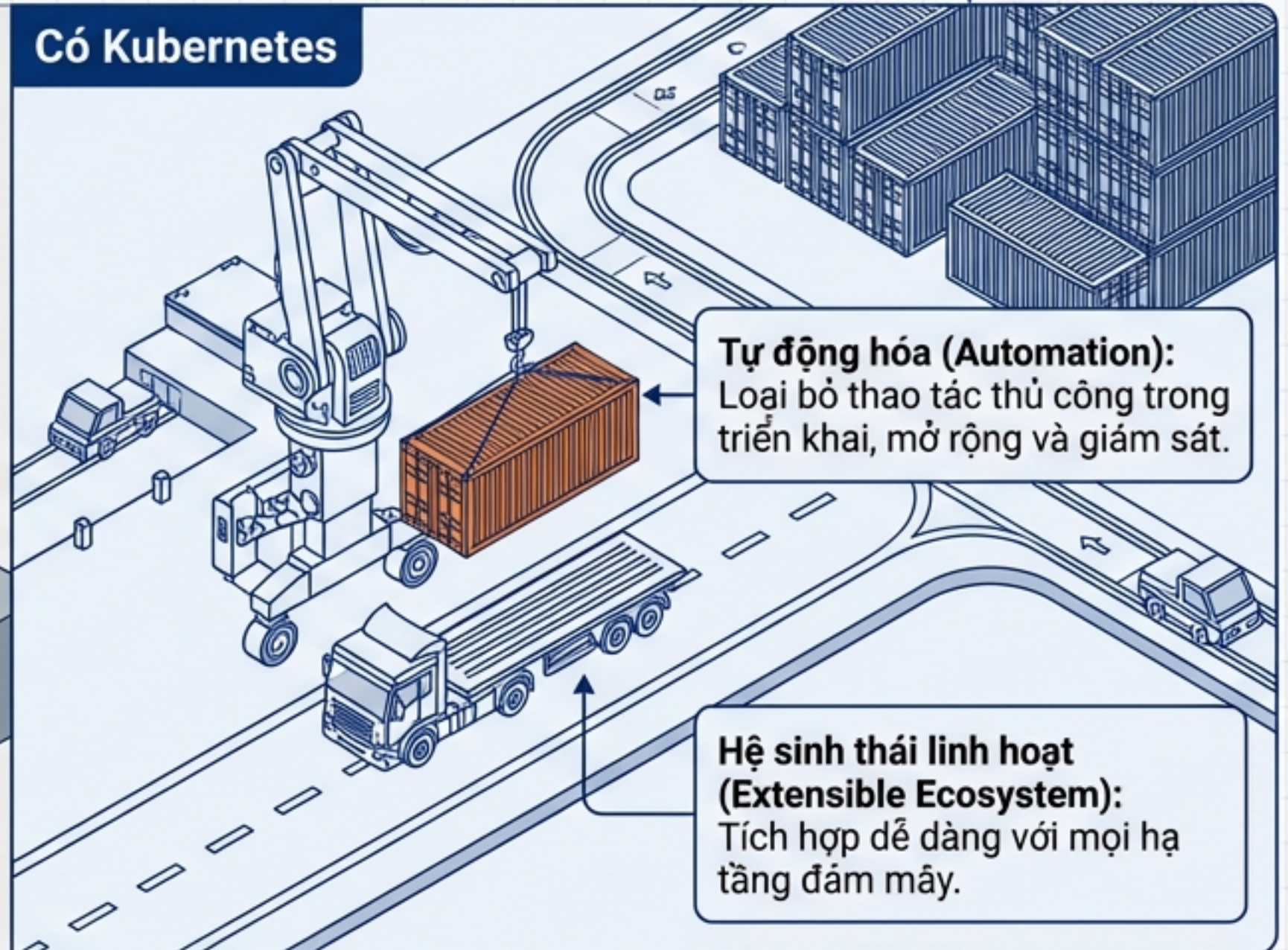
Chuyển đổi từ Hỗn loạn sang Kỷ luật Tự động hóa

Kubernetes không chỉ chạy các ứng dụng; nó tự động hóa toàn bộ vòng đời của chúng. Đây là một nền tảng mã nguồn mở, di động và có thể mở rộng, chuyên quản lý các khối lượng công việc (workloads) và dịch vụ được container hóa.

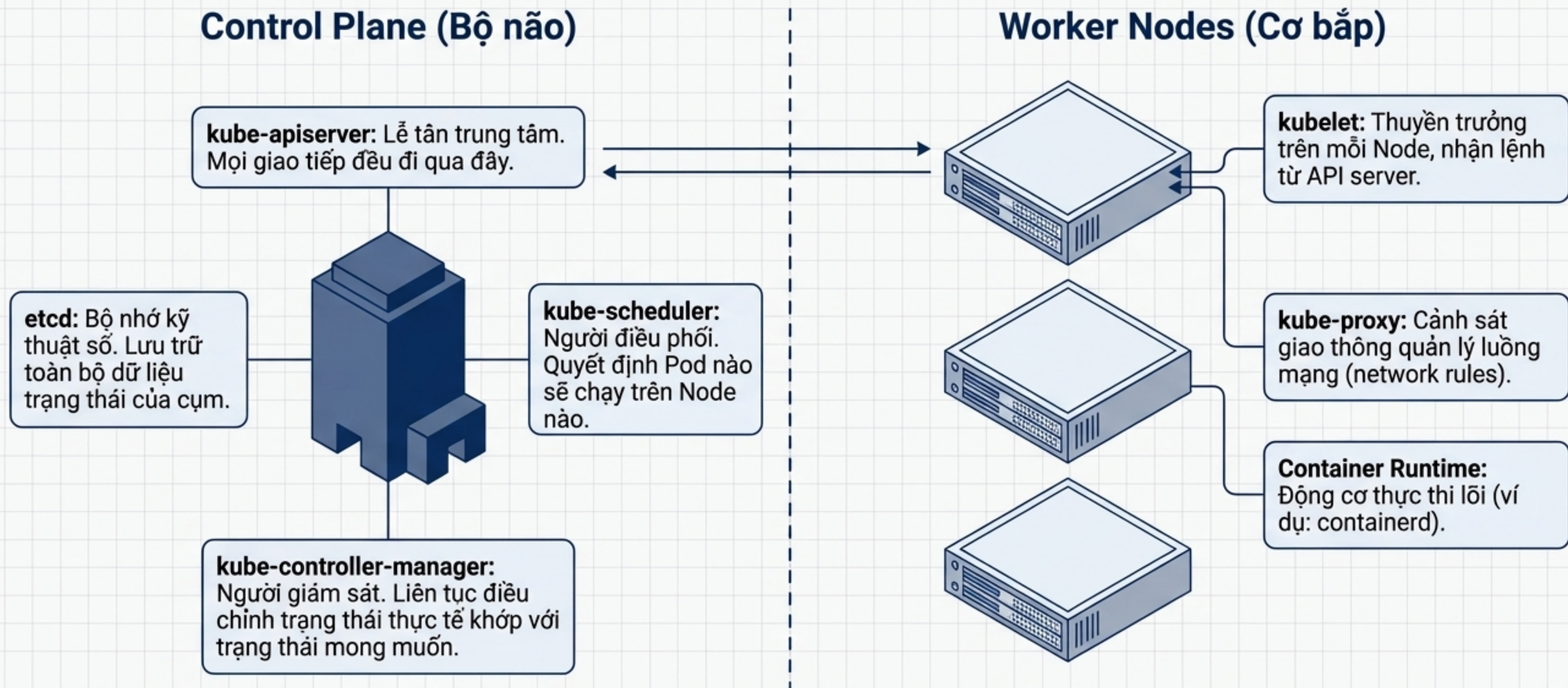
Mô hình cũ



Có Kubernetes

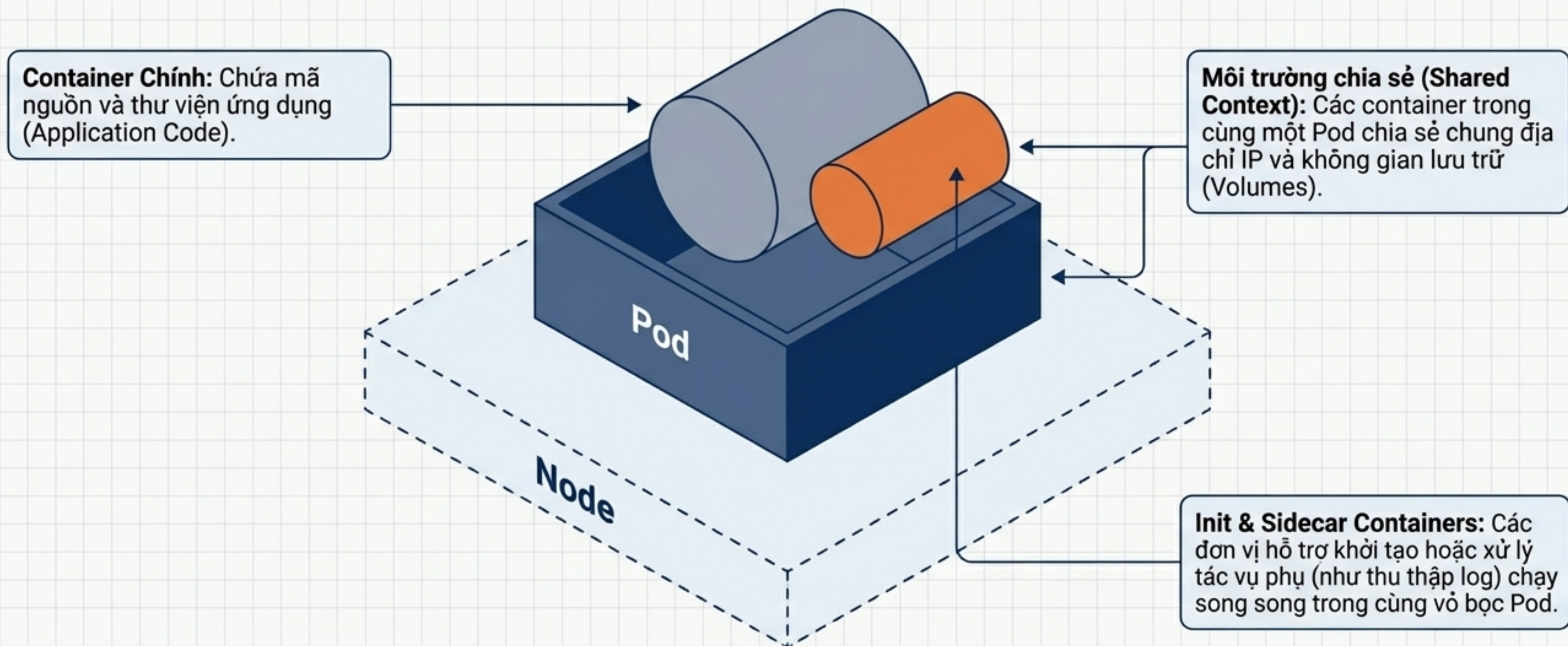


Cấu trúc Phân tách: Bộ não và Cơ bắp của Cụm



Đơn vị Vi mô: Pods và Vùng chứa

Kubernetes không trực tiếp quản lý từng Container. Khối xây dựng cơ bản nhất là **Pod** — một lớp vỏ bọc logic chứa một hoặc nhiều Container chia sẻ chung tài nguyên mạng và lưu trữ.



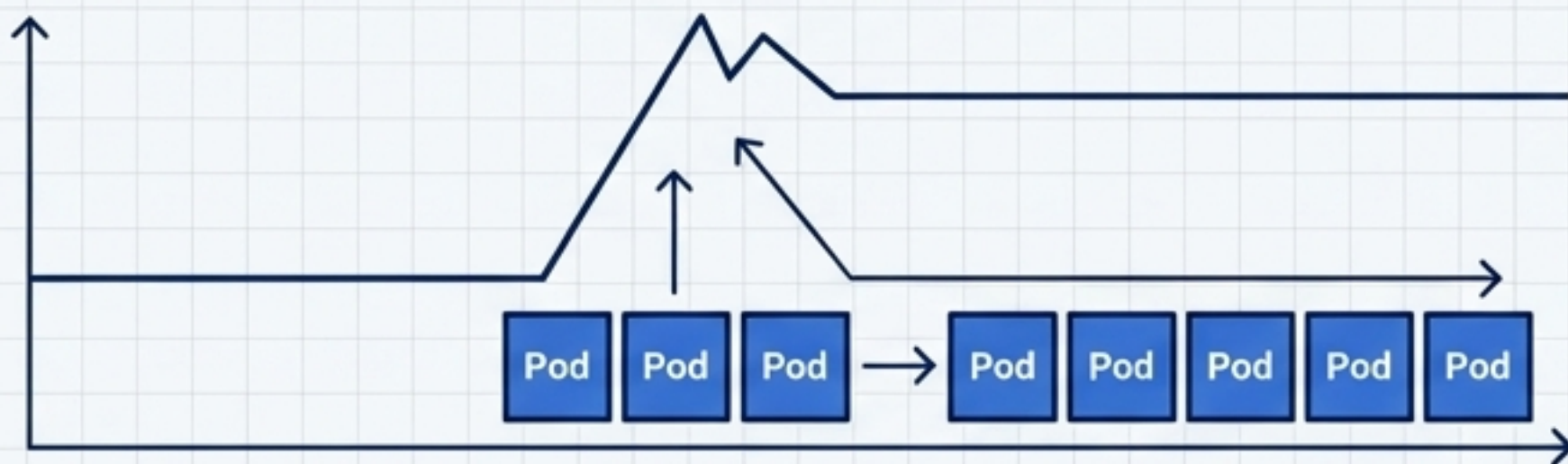
Ma trận Lựa chọn Khối lượng Công việc (Workloads)

			
Deployments	StatefulSets	DaemonSets	Jobs & CronJobs
Đặc tính: Vô trạng thái (Stateless). Có thể thay thế dễ dàng.	Đặc tính: Có trạng thái (Stateful). Định danh mạng và lưu trữ cố định.	Đặc tính: Tác vụ chạy ngầm. Đảm bảo chạy đúng một bản sao trên mọi Node.	Đặc tính: Tác vụ có điểm kết thúc.
Mục đích: Đảm bảo số lượng bản sao (replicas) luôn chạy ổn định.	Mục đích: Quản lý các ứng dụng cần ghi nhớ dữ liệu sau khi khởi động lại.	Mục đích: Cung cấp dịch vụ nền cho cơ sở hạ tầng.	Mục đích: Thực thi các quy trình xử lý lô (batch processing) hoặc chạy theo lịch trình (Cron).
Ứng dụng: Web servers, API backends.	Ứng dụng: Cơ sở dữ liệu (MySQL, Cassandra), ZooKeeper.	Ứng dụng: Thu thập log, giám sát tài nguyên (Prometheus node-exporter).	Ứng dụng: Sao lưu dữ liệu định kỳ, kết xuất báo cáo.

Hệ Miễn dịch Kỹ thuật số: Mở rộng và Tự phục hồi

Sức mạnh cốt lõi của Kubernetes nằm ở khả năng tự động hóa việc duy trì tính khả dụng cao (High Availability) mà không cần sự can thiệp của con người.

Tự động mở rộng (Autoscaling)



HPA (Horizontal Pod Autoscaler): Tự động tăng/giảm số lượng Pod dựa trên mức sử dụng CPU/Memory hoặc các số đo tùy chỉnh.

Thăm dò & Phục hồi (Probes & Healing)

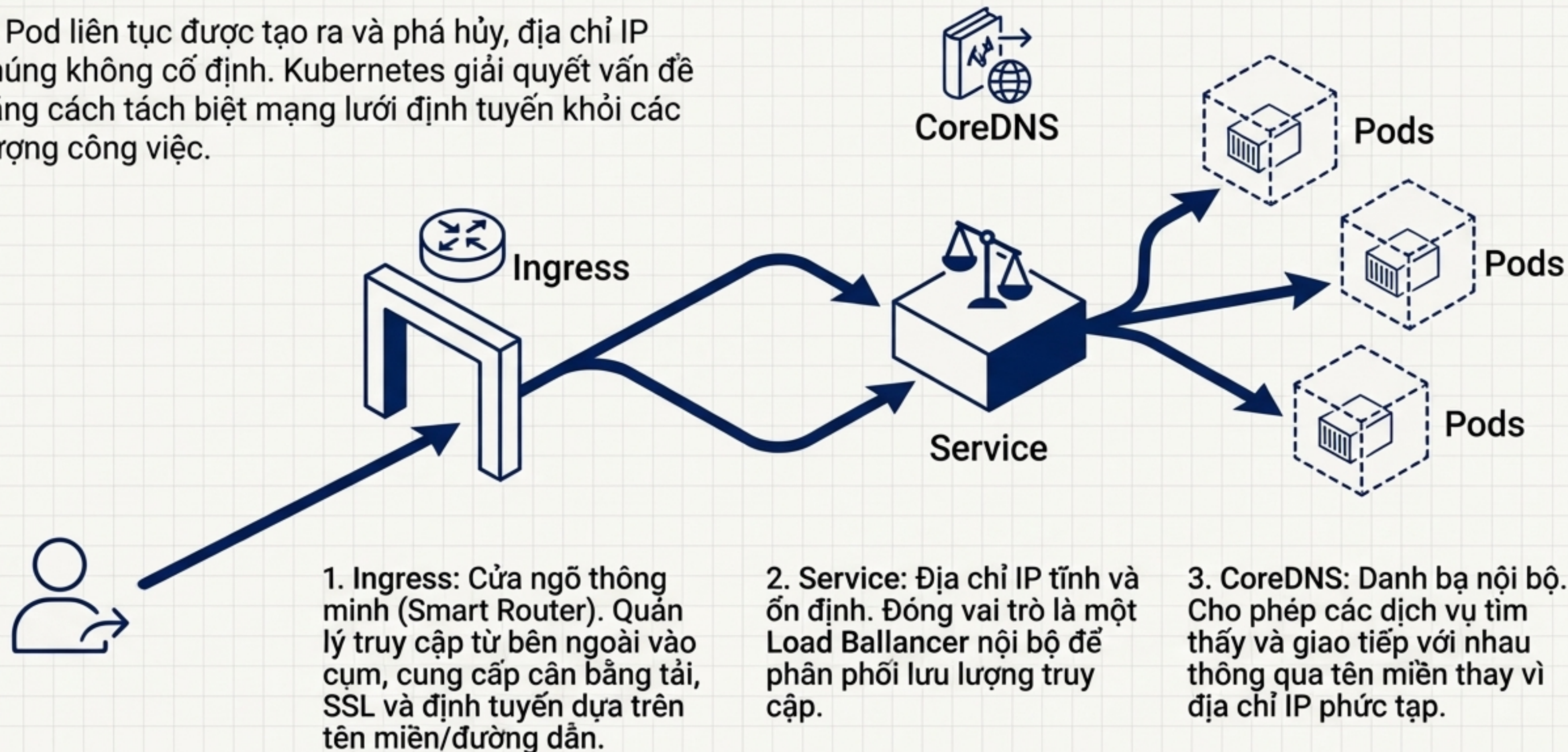


Liveness Probe: Kiểm tra xem ứng dụng còn sống không. Nếu không, khởi động lại Pod.

Readiness Probe: Kiểm tra xem ứng dụng đã sẵn sàng nhận traffic chưa. Tránh chuyển hướng người dùng vào Pod đang khởi động.

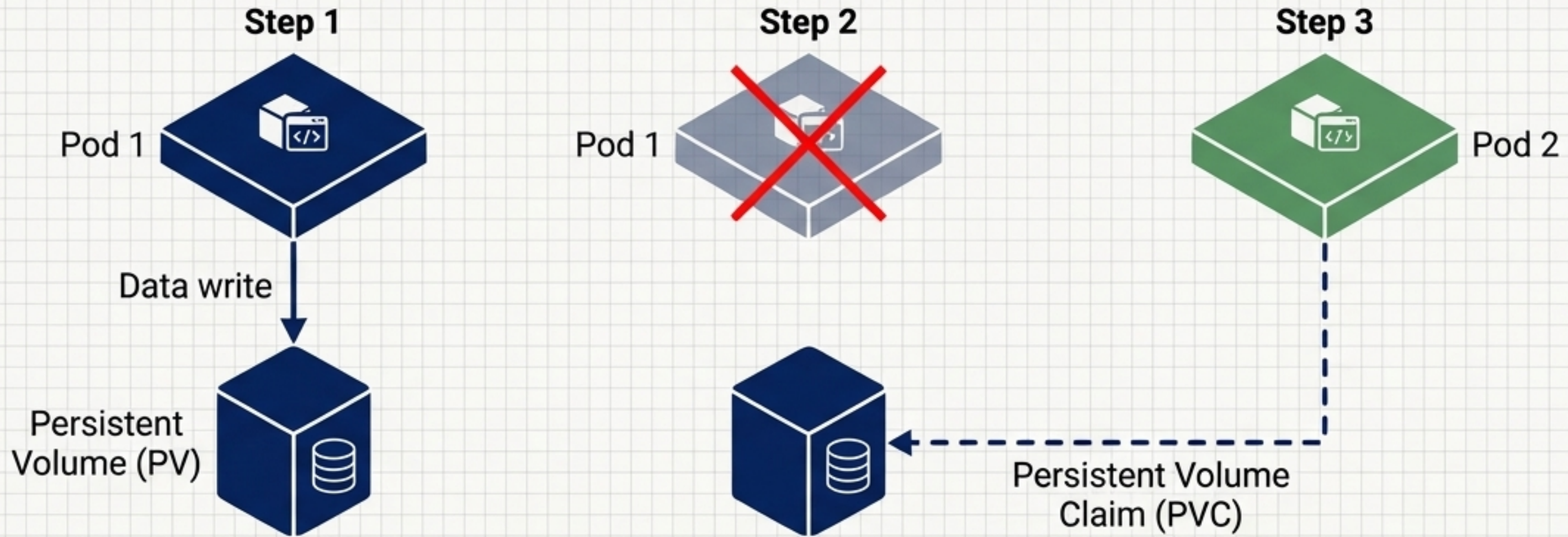
Hệ thống Giao thông: Định tuyến và Khám phá Dịch vụ

Vì các Pod liên tục được tạo ra và phá hủy, địa chỉ IP của chúng không cố định. Kubernetes giải quyết vấn đề này bằng cách tách biệt mạng lưới định tuyến khỏi các khối lượng công việc.



Quản lý Trạng thái: Lưu trữ Bền vững

Kiến trúc của Kubernetes mặc định là vô trạng thái (ephemeral). Để dữ liệu tồn tại sau vòng đời của một Pod, hệ thống tách biệt phần cứng lưu trữ khỏi ứng dụng.



Tạm thời (Ephemeral Volumes)



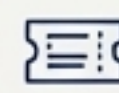
Gắn liền với vòng đời Pod. Dữ liệu bị xóa sạch khi Pod bị hủy.

Bền vững (Persistent Volumes - PV)



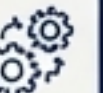
Khối dung lượng lưu trữ độc lập trên hạ tầng. Tồn tại vĩnh viễn cho đến khi bị xóa chủ động.

Yêu cầu (PVC)



Tấm vé yêu cầu cấp phát dung lượng của ứng dụng. K8s sẽ tự động khớp PVC với PV phù hợp.

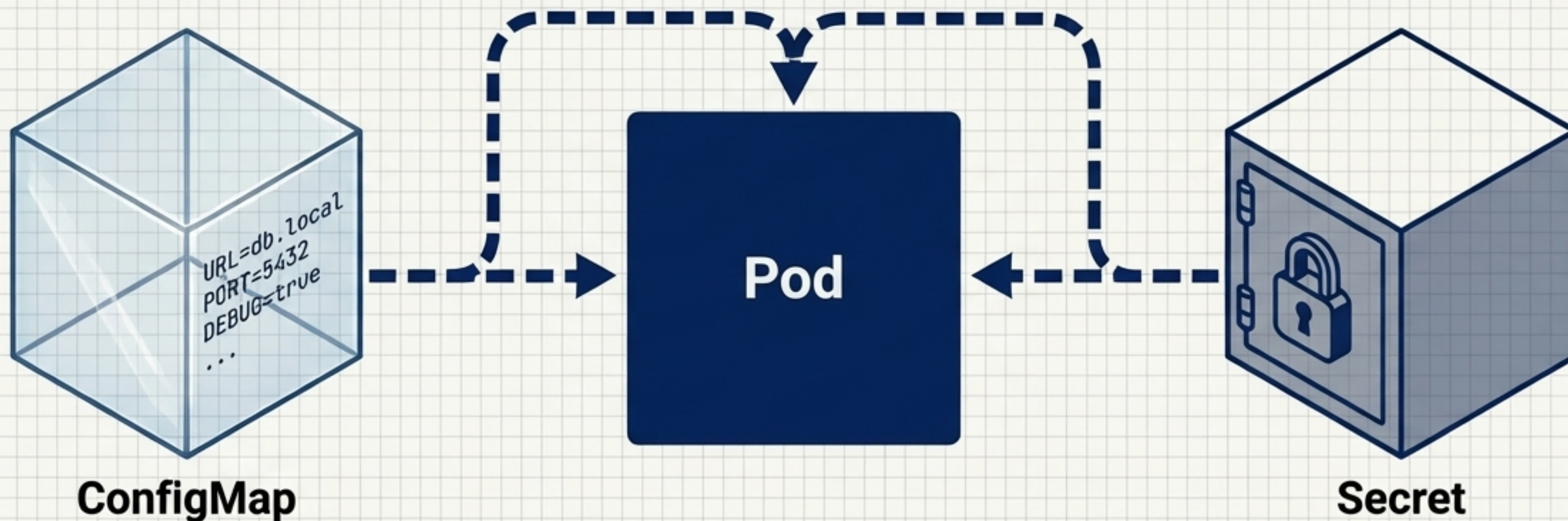
Storage Classes



Khung cấp phát tự động (Dynamic Provisioning). Tự động tạo ổ cứng mới theo yêu cầu hình.

Bơm Ngữ cảnh: Tách biệt Cấu hình và Mã nguồn

Theo nguyên tắc của ứng dụng Cloud-Native, hình ảnh container (Image) không bao giờ được chứa cấu hình cứng. Kubernetes cho phép bạn "tiêm" cấu hình vào môi trường chạy theo thời gian thực.

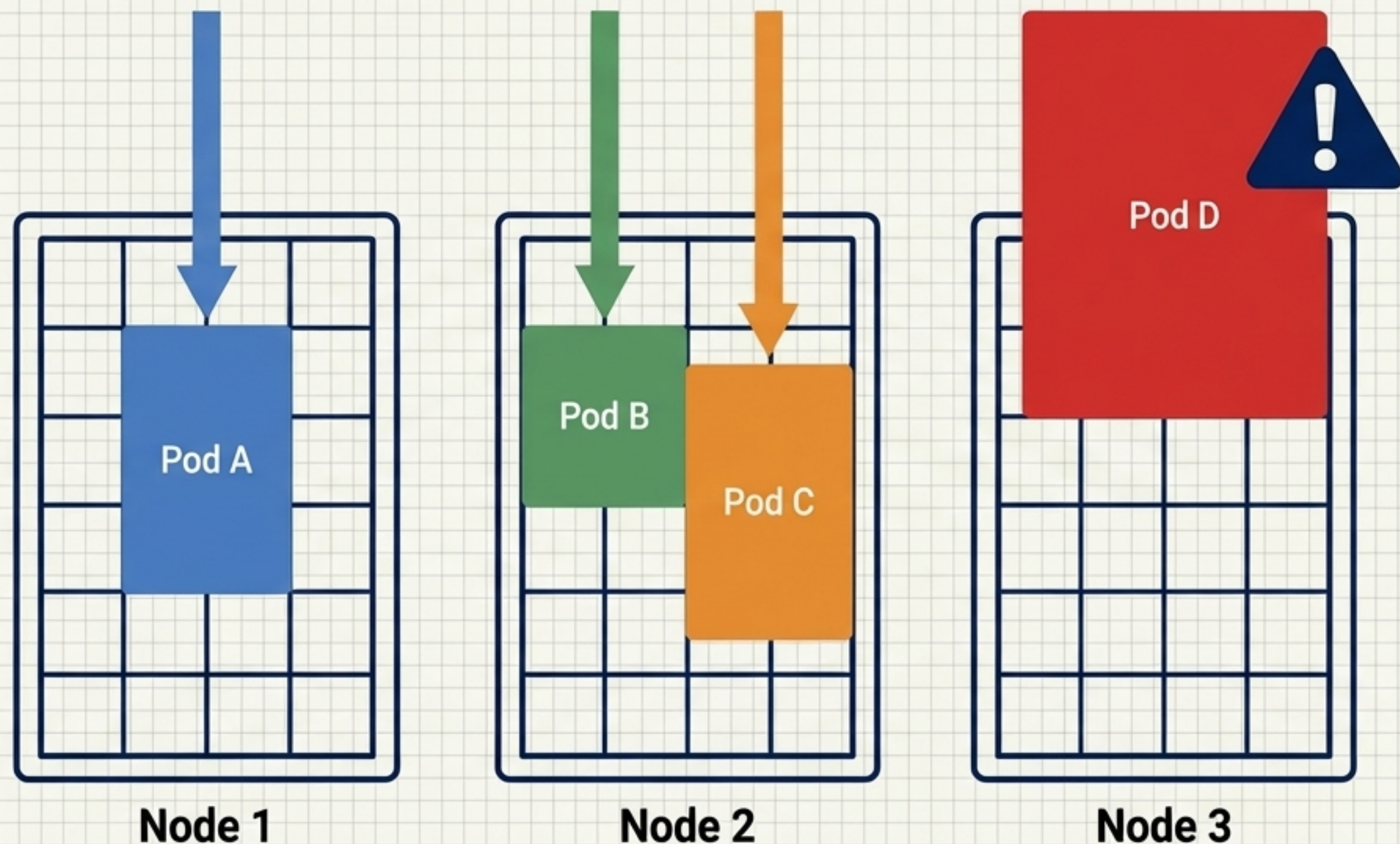


ConfigMaps (Biến môi trường & Tập cấu hình):
Dùng để lưu trữ dữ liệu dạng văn bản thuần túy. Ví dụ: URL cơ sở dữ liệu, tham số khởi động, cờ tính năng (feature flags).

Secrets (Dữ liệu Nhạy cảm): Tương tự ConfigMap nhưng được thiết kế để chứa thông tin bảo mật (Mật khẩu, SSH keys, OAuth tokens). Dữ liệu được mã hóa ở dạng Base64 và có thể được mã hóa tĩnh (encrypted at rest) bằng KMS.

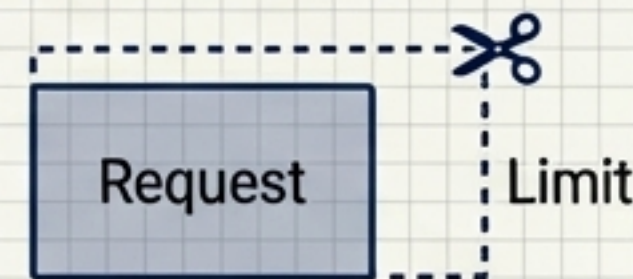
Thuật toán Lập lịch: Chơi Tetris với Tài nguyên

kube-scheduler hoạt động như một cỗ máy xếp hình khổng lồ, liên tục đánh giá dung lượng Node và kích thước Pod để phân bổ tài nguyên tối ưu nhất.



Requests vs. Limits

Request là mức tối thiểu được bảo đảm (diện tích khối block). Limit là mức tối đa được phép sử dụng trước khi bị hệ thống cắt (throttling) hoặc tiêu diệt (OOMKilled).



Taints & Tolerations

Đánh dấu Node (Taint) để 'đẩy lùi' các Pod không mong muốn. Pod chỉ được xếp vào nếu có 'Toleration' (miễn dịch) tương ứng.



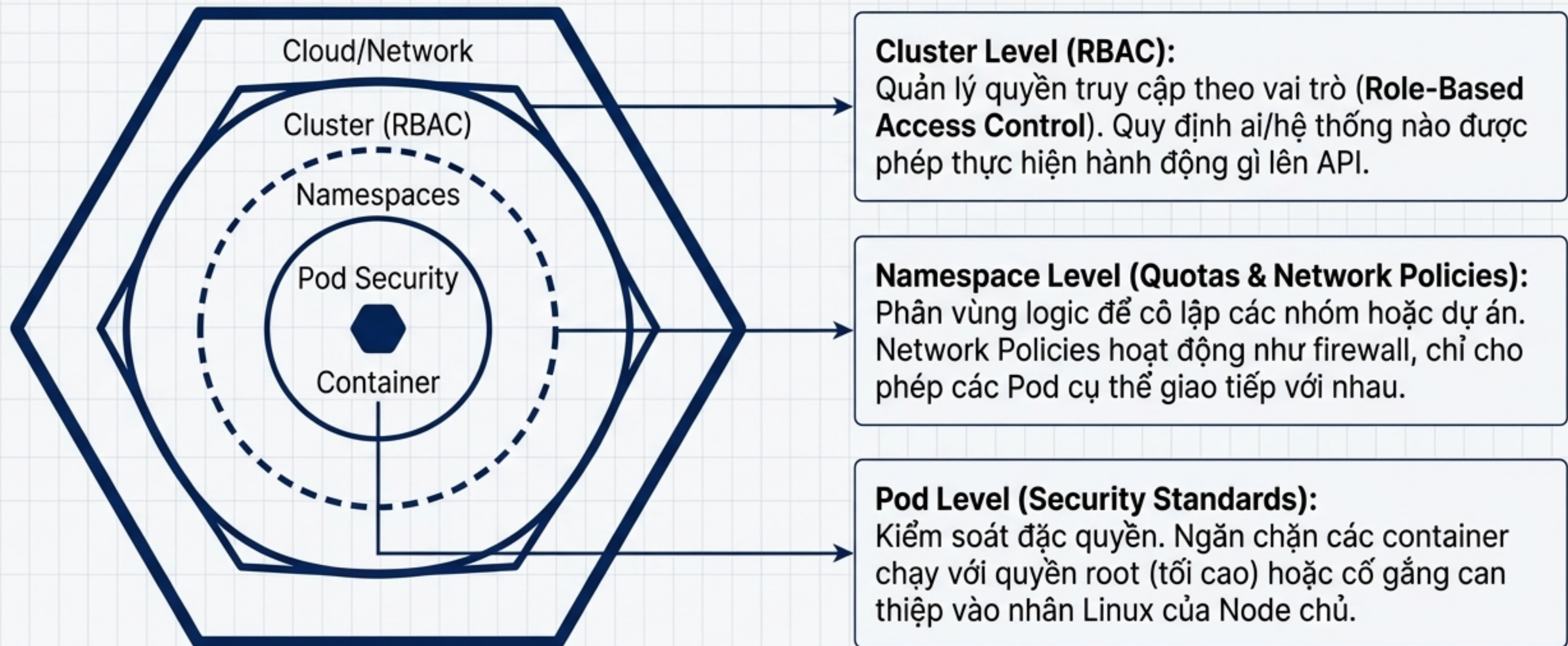
Node Affinity

Lực hút nam châm. Chỉ định Pod phải (hoặc nên) được đặt trên các Node có nhãn (label) cụ thể, ví dụ như Node có GPU.



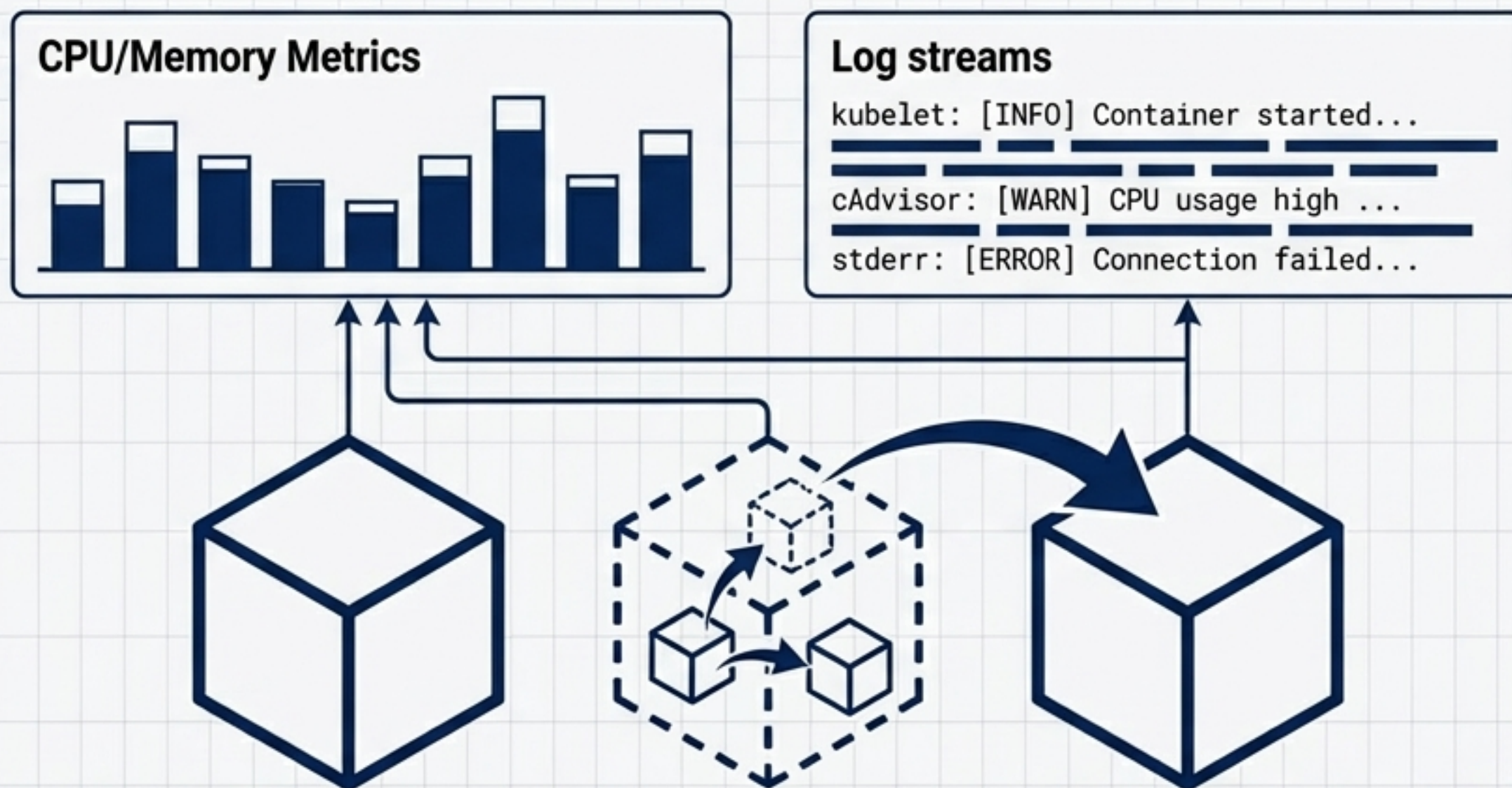
Vành đai An ninh: Kiểm soát và Giới hạn Quyền truy cập

Bảo mật trong Kubernetes yêu cầu phương pháp tiếp cận phòng thủ chiều sâu (defense-in-depth), thiết lập rào chắn từ mức độ cụm cho đến tận bên trong container.



Khả năng Quan sát và Quản trị Cụm

Để quản lý hàng ngàn vùng chứa dịch chuyển liên tục, quản trị viên cần hệ thống thu thập số liệu phân tán và công cụ quản trị vòng đời Node an toàn.



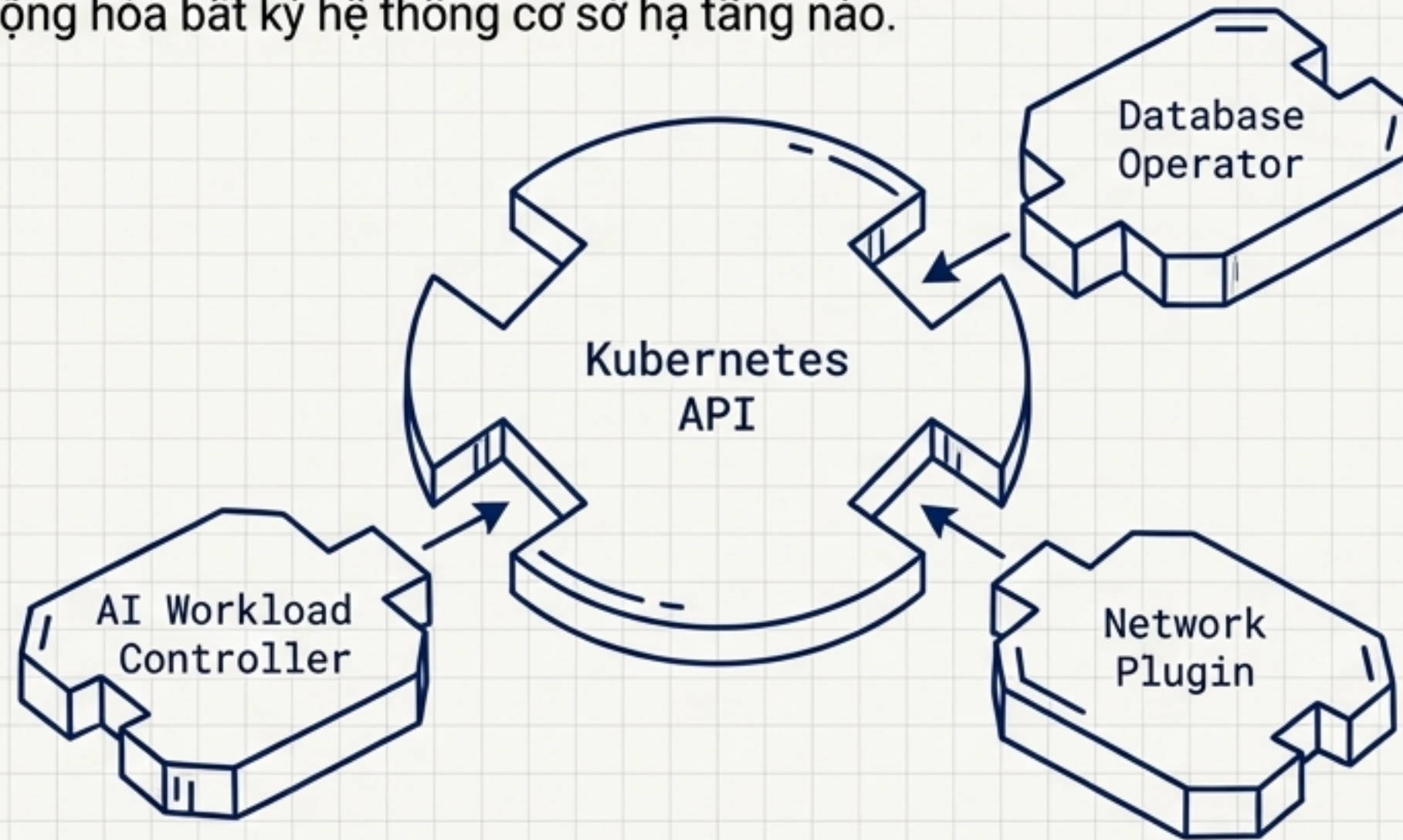
Metrics (Chỉ số đo lường): Dữ liệu hiệu suất hệ thống từ kubelet và cAdvisor để cảnh báo áp lực tài nguyên (CPU, RAM).

Logging Architecture: Thu thập luồng đầu ra (stdout/stderr) từ từng container và chuyển tiếp đến các hệ thống lưu trữ log tập trung.

Bảo trì An toàn (Safely Drain Nodes): Trục xuất một cách an toàn tất cả Pod khỏi một Node để bảo trì/nâng cấp hệ điều hành mà không làm gián đoạn dịch vụ.

Mở rộng Kubernetes: Các Mô-đun Tùy chỉnh (CRD & Operators)

Kubernetes không chỉ giới hạn ở các đối tượng mặc định (Pod, Service). Nó là một bộ khung có thể được mở rộng không giới hạn để tự động hóa bất kỳ hệ thống cơ sở hạ tầng nào.



CRD (Custom Resource Definitions):

Cho phép bạn tự định nghĩa các đối tượng cấu hình hoàn toàn mới trong API của Kubernetes (ví dụ: đối tượng PostgreSQLDatabase).

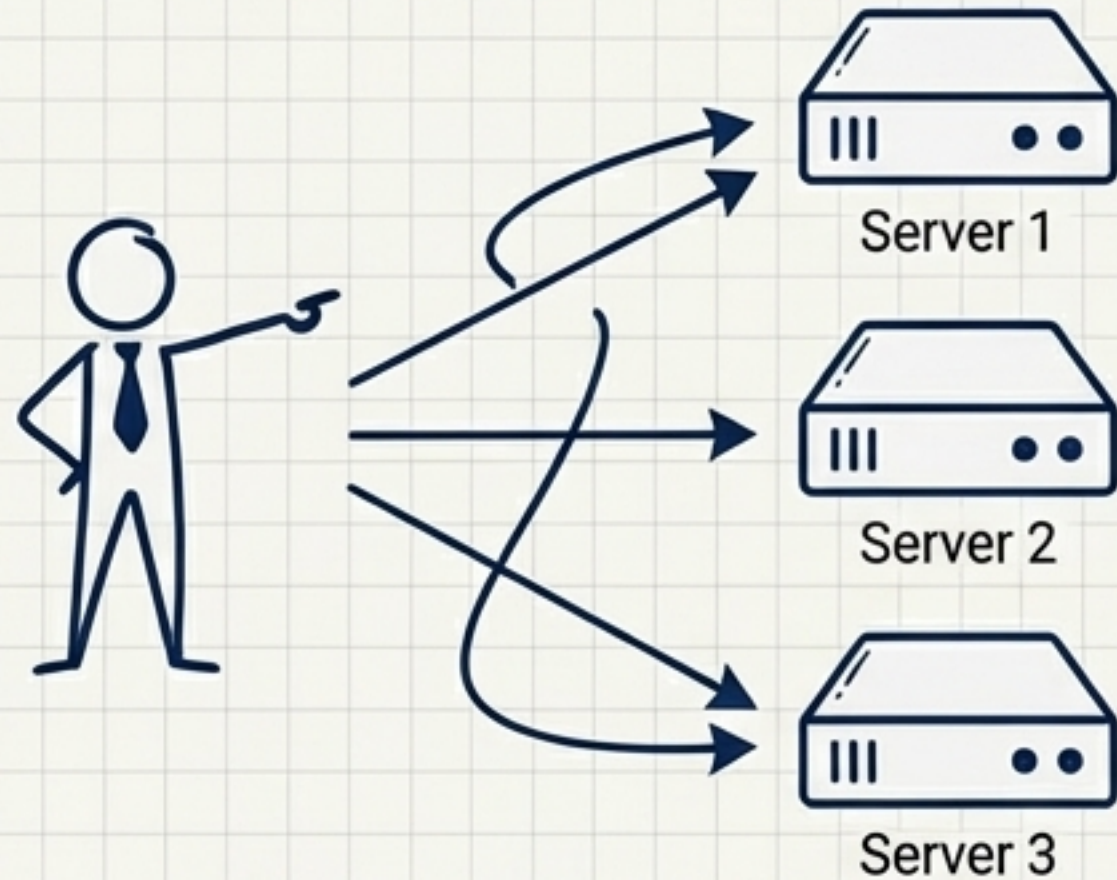
The Operator Pattern:

Dạy cho Kubernetes cách quản lý các ứng dụng phức tạp. Một đoạn mã tùy chỉnh đóng vai trò như một "chuyên gia con người ảo", liên tục theo dõi, sao lưu và sửa lỗi tự động cho ứng dụng của bên thứ ba.

Giao diện Giao tiếp: kubectl và Quản lý Khai báo

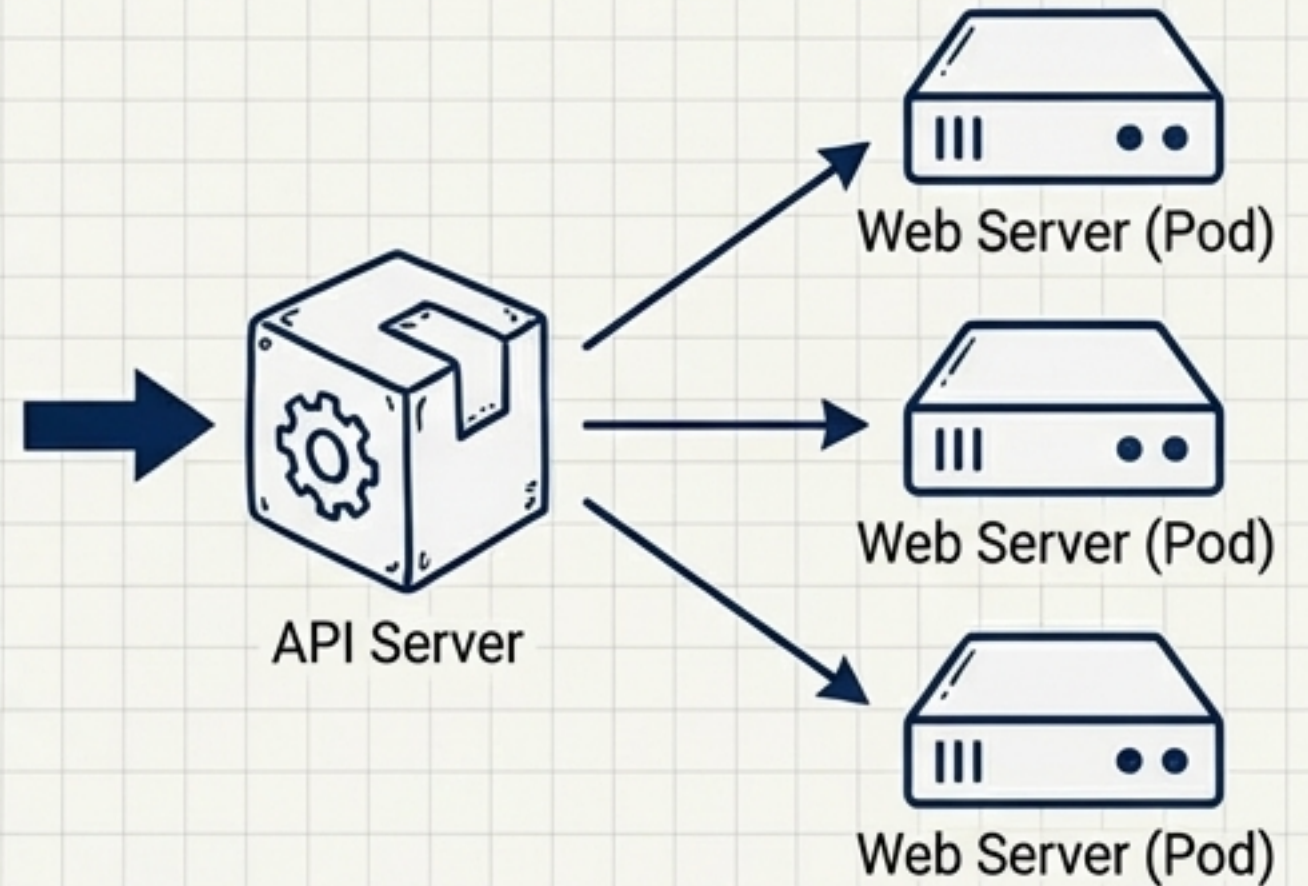
Người dùng tương tác với cụm thông qua công cụ dòng lệnh kubectl, nhưng sức mạnh thực sự nằm ở sự dịch chuyển tư duy từ 'ra lệnh' sang 'khai báo trạng thái'.

Cách cũ (Imperative): Ra lệnh từng bước.



Kubernetes (Declarative): Khai báo kết quả.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```



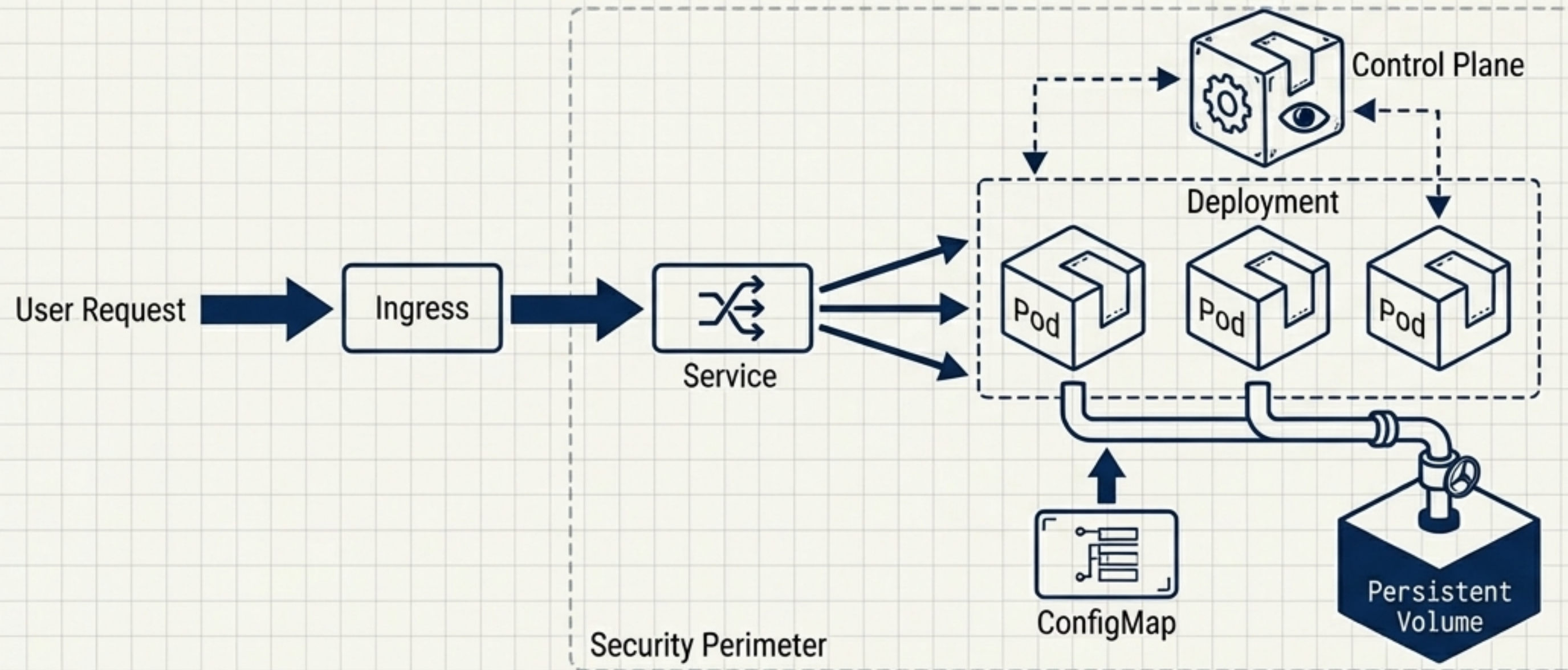
- **Quản lý Khai báo (Declarative):** Bạn cung cấp một tệp cấu hình (thường là YAML) miêu tả kết quả cuối cùng (VD: Tôi cần 3 web servers).

- **Bộ điều khiển (Controllers):** Kubernetes đọc tệp YAML và tự suy luận ra các bước cần thiết để tạo ra, theo dõi và duy trì trạng thái đó mãi mãi.

- **Server-Side Apply:** Nền tảng tự động xử lý xung đột và cập nhật mượt mà khi người dùng thay đổi tệp cấu hình theo thời gian.

Bản thiết kế Toàn cảnh: Khung Vận hành Đám mây

Tóm tắt Hệ sinh thái. Kubernetes kết nối mạng lưới giao thông thông minh, quản lý trạng thái lưu trữ bền vững, tự động hóa vòng đời tính toán và thực thi bảo mật nghiêm ngặt.



Lộ trình Mạng:
Ingress -> Service
-> Pods

Quản lý Khối lượng:
Control Plane ->
Deployment -> Pod

Quản lý Ngữ cảnh:
ConfigMap/Secrets
tiêm dữ liệu

Trạng thái Dữ liệu:
Pod kết nối PV
qua PVC

Kubernetes cung cấp bản thiết kế kỹ thuật số nền tảng để xây dựng các ứng dụng linh hoạt, tự phục hồi và mở rộng vô hạn.